



Unidade 04 - Lista de Exercícios 1

1. Um comerciante deseja monitorar o faturamento diário de sua loja durante uma semana (7 dias). Desenvolva um sistema que utilize um vetor para armazenar esses valores e ofereça um menu interativo para análise dos dados.

Entrada:

- Opção do menu (1: Cadastrar, 2: Analisar, 3: Pesquisar, 0: Sair).
- Valor do faturamento para cada um dos 7 dias.
- Valor de meta diária para pesquisa.

Importante:

- (a) O programa deve rodar em um laço **while** contínuo até que a opção 0 seja escolhida.
- (b) Na opção 1, utilize um laço **for** para preencher um vetor de tamanho 7. Valide cada entrada: se o valor for negativo, use um **while** para forçar a reentrada correta.
- (c) Na opção 2, calcule a média semanal e identifique quantos dias ficaram acima da média usando laços aninhados ou sequenciais.
- (d) Na opção 3, verifique se houve algum dia com faturamento exatamente igual à meta informada.

Definição Matemática:

$$\text{Média} = \frac{\sum_{i=0}^6 \text{Faturamento}_i}{7}$$

Exemplo de Saída:

```
—— MENU ——  
1. Cadastrar  
2. Analisar  
0. Sair  
Opcao: 1  
Dia 1: 150.0  
Dia 2: -10 (Erro! Digite valor positivo)  
Dia 2: 200.0  
...
```



2. Uma escola técnica precisa gerenciar as notas de 3 alunos em 3 avaliações diferentes. Crie um programa que utilize uma matriz 3×3 para armazenar essas notas e processar os resultados individuais e coletivos.

Entrada:

- Notas dos alunos (armazenadas em uma matriz `notas[3][3]`).
- Menu de seleção (1: Lançar Notas, 2: Boletim Geral, 0: Sair).

Importante:

- (a) Use laços de repetição encadeados (`for` dentro de `for`) para percorrer a matriz.
- (b) Para cada nota inserida, valide se está no intervalo $[0, 10]$. Caso contrário, exiba erro e peça novamente.
- (c) O boletim deve mostrar a média de cada aluno. Se a média for ≥ 7 , exiba "Aprovado"; se ≥ 5 e < 7 , "Recuperação"; caso contrário, "Reprovado".
- (d) Implemente um contador para verificar quantos alunos no total ficaram em "Recuperação".

Exemplo de Saída:

Aluno 1, Nota 1: 8.5

Aluno 1, Nota 2: 7.0

...

Resultado Aluno 1: Media 7.75 – APROVADO

3. Um laboratório de botânica monitora o crescimento de 5 plantas em um experimento. Os dados de altura (em cm) devem ser armazenados em um vetor e comparados com uma altura de referência.

Entrada:

- Vetor de alturas das 5 plantas.
- Altura de referência para crescimento esperado.

Importante:

- (a) O menu deve permitir: 1. Inserir dados, 2. Relatório de Crescimento, 0. Sair.
- (b) No relatório, use um laço para comparar cada planta com a referência. Calcule a diferença percentual de cada uma.
- (c) Utilize laços aninhados para simular um processo de "estabilização" de dados, onde cada valor é comparado com os demais para encontrar a planta mais alta.
- (d) Valide para que nenhuma planta tenha altura zero ou negativa.

Definição Matemática:

$$\text{Diferença \%} = \left(\frac{\text{Altura} - \text{Referência}}{\text{Referência}} \right) \times 100$$



4. Um sistema de controle de estoque de uma farmácia precisa gerenciar 2 prateleiras, cada uma com 4 tipos de medicamentos. Utilize uma matriz 2×4 para controlar a quantidade de caixas disponíveis.

Entrada:

- Quantidade de caixas para cada posição da matriz.
- Opção de venda (solicitar prateleira e coluna).

Importante:

- (a) O programa deve iniciar com o preenchimento da matriz (validação: quantidade não negativa).
- (b) No menu de venda, se o usuário solicitar uma quantidade maior do que a disponível em `estoque[p][c]`, exiba "Estoque Insuficiente".
- (c) Use um laço `for` para mostrar a matriz formatada na tela após cada operação.
- (d) Calcule o total de itens em falta (quantidade igual a zero) usando condicionais dentro dos laços.

5. Um atleta de alto rendimento registra seus tempos (em segundos) em 5 voltas de um circuito. O sistema deve armazenar os tempos em um vetor e realizar uma análise de consistência.

Entrada:

- Vetor de tempos (5 posições).
- Opção para verificar "Volta Mais Rápida" ou "Média de Ritmo".

Importante:

- (a) Implemente a validação: tempos menores que 10 segundos devem ser descartados como erro de sensor (peça nova entrada).
- (b) Use um laço `for` para encontrar o menor valor dentro do vetor (volta mais rápida).
- (c) Utilize um laço aninhado para comparar cada volta com todas as outras, calculando a variância simples entre elas.
- (d) O menu deve permitir múltiplas análises até o encerramento.



6. Uma rede de cinemas possui uma pequena sala VIP com 3 fileiras e 4 poltronas cada. Crie um mapa de ocupação usando uma matriz de inteiros (0 para livre, 1 para ocupado).

Entrada:

- Coordenadas do assento (Fila e Poltrona).
- Menu: 1. Ver Mapa, 2. Reservar, 0. Sair.

Importante:

- (a) Inicialize a matriz com zeros usando laços aninhados.
- (b) Ao reservar, verifique se a posição `mapa[f][p]` já é 1. Se sim, informe "Assento Ocupado" e peça outra escolha.
- (c) O mapa deve ser exibido usando um laço `for` que imprime 'L' para 0 e 'X' para 1.
- (d) Calcule a porcentagem de ocupação da sala a cada reserva.

7. Um sistema de meteorologia registra a temperatura de 3 cidades em 2 horários do dia (manhã e tarde). Use uma matriz 3×2 para organizar esses dados.

Entrada:

- Temperaturas (matriz `temp[3][2]`).
- Menu de consulta.

Importante:

- (a) Valide as temperaturas: valores abaixo de -50 ou acima de 60 graus devem ser considerados erro de hardware.
- (b) Calcule a média por cidade e exiba um alerta "Risco de Geada" se a temperatura da manhã for < 0 .
- (c) Use laços aninhados para encontrar a maior temperatura registrada em todo o sistema.
- (d) Permita que o usuário atualize a temperatura de uma cidade específica informando os índices.



8. Um banco de sangue precisa controlar o estoque de 4 tipos sanguíneos (A, B, AB, O) em 2 hospitais parceiros através de uma matriz 2×4 .

Entrada:

- Quantidade de bolsas por tipo/hospital.
- Valor de alerta crítico (limite mínimo).

Importante:

- (a) O programa deve solicitar a quantidade de bolsas, rejeitando entradas negativas.
- (b) O sistema deve percorrer a matriz e listar quais hospitais estão com estoque abaixo do limite crítico para cada tipo.
- (c) Calcule o total de bolsas do tipo 'O' somando as posições específicas da matriz em ambos os hospitais.
- (d) Ofereça um menu para atualizar o estoque de uma remessa recebida.

9. Um pequeno estacionamento possui 5 vagas em linha (vetor). O sistema controla o tempo de permanência de cada veículo em minutos.

Entrada:

- Tempo de permanência por vaga (0 se vazia).
- Valor da hora de estacionamento.

Importante:

- (a) Calcule o valor a ser pago para cada vaga ocupada (considere divisões de frações de hora).
- (b) Use um laço para encontrar qual veículo ficou mais tempo estacionado.
- (c) Valide a entrada do valor da hora (deve ser positivo).
- (d) Implemente um menu que permita ver o faturamento total acumulado do dia.



10. Uma nutricionista deseja registrar o consumo de calorias de 3 pacientes durante as 4 refeições principais do dia (matriz 3×4).

Entrada:

- Calorias por refeição/paciente.
- Limite diário de calorias recomendado.

Importante:

- (a) Percorra a matriz usando laços aninhados para somar o consumo total de cada paciente.
 - (b) Utilize condicionais para comparar a soma total com o limite recomendado e exibir "Dieta OK" ou "Excesso de Calorias".
 - (c) Identifique qual foi a refeição mais calórica em média entre todos os pacientes (coluna da matriz com maior soma).
 - (d) Valide se as calorias informadas são maiores que zero.
11. Um setor de Recursos Humanos deseja monitorar os minutos de atraso de 5 funcionários durante um dia de auditoria. Desenvolva um sistema que armazene esses dados em um vetor e permita o gerenciamento através de um menu.

Entrada:

- Nome do funcionário e minutos de atraso.
- Opção do menu (1: Registrar, 2: Relatório Crítico, 0: Sair).

Importante:

- (a) O sistema deve validar a entrada: minutos não podem ser negativos. Use um laço **while** para garantir a reentrada.
- (b) No relatório, utilize um laço **for** para percorrer o vetor e aplique a regra: se atraso > 15 minutos, aplique desconto proporcional; se > 30 , exiba "Alerta de Advertência".
- (c) Utilize um laço aninhado para comparar o atraso de cada funcionário com a média de todos os outros para identificar padrões de comportamento.
- (d) Calcule o total de descontos aplicados usando operadores aritméticos.

Definição Matemática:

$$\text{Desconto} = \frac{\text{Atraso}}{60} \times \frac{\text{Salário Hora}}{2}$$



12. Uma empresa de logística controla a carga (em toneladas) de 3 caminhões em 3 rotas diferentes. Utilize uma matriz 3×3 para gerenciar essas informações.

Entrada:

- Cargas para a matriz logística[3][3].
- Menu: 1. Carregar Matriz, 2. Verificar Sobrecarga, 0. Sair.

Importante:

- (a) Use laços encadeados para preencher e percorrer a matriz.
- (b) Valide cada carga: se for superior a 10 toneladas, emita um erro e peça um novo valor usando `while`.
- (c) No relatório, calcule a soma total por caminhão (linha) e por rota (coluna).
- (d) Verifique se alguma rota específica ultrapassou o limite de 25 toneladas no total dos 3 caminhões.

Exemplo de Saída:

```
Caminhao 1, Rota 1: 8.5
Caminhao 1, Rota 2: 12.0 (Erro! Carga maxima 10t)
...
Total Rota 1: 22.0 t – DENTRO DO LIMITE
```

13. Em um torneio de E-sports, 4 equipes disputam 3 partidas. As pontuações de cada partida devem ser organizadas em uma matriz 4×3 .

Entrada:

- Pontuação das equipes em cada partida.
- Opção do menu para ver o Ranking.

Importante:

- (a) O menu deve oferecer: 1. Inserir Pontos, 2. Ranking Final, 0. Sair.
- (b) Valide os pontos: devem ser sempre positivos.
- (c) Use laços aninhados para calcular a soma total de pontos de cada equipe e determinar qual equipe teve a maior pontuação em uma única partida.
- (d) Se a pontuação total for > 500 , aplique um multiplicador de "Bônus de Vitória" de 1.1x.



14. Um influenciador digital deseja analisar o engajamento de seus últimos 6 posts. Desenvolva um programa que utilize um vetor para armazenar o número de curtidas de cada post.

Entrada:

- Vetor de curtidas (6 posições).
- Meta de engajamento desejada.

Importante:

- (a) O sistema deve rodar em um laço **while**.
- (b) Na opção de análise, use um **for** para identificar "Posts Virais" (curtidas $> 2 \times$ a média).
- (c) Utilize um laço aninhado para ordenar os posts do mais curtido para o menos curtido (simulação simples de ordenação).
- (d) Caso a meta não seja atingida em nenhum post, exiba uma mensagem de "Revisar Conteúdo".

15. Um sistema de agricultura inteligente monitora a umidade de um solo dividido em um grid de 4×4 sensores. Use uma matriz para representar esse campo.

Entrada:

- Percentual de umidade de cada sensor (0 a 100).

Importante:

- (a) Use laços aninhados para inicializar o grid.
- (b) Valide cada leitura: se for < 0 ou > 100 , use um **while** para corrigir o dado.
- (c) O sistema deve identificar áreas secas (umidade $< 30\%$) e sugerir ativação de irrigação apenas para aquelas coordenadas $[i][j]$.
- (d) Calcule a média de umidade de cada quadrante da matriz.



16. Uma transportadora monitora o consumo de combustível (km/l) de sua frota de 5 veículos leves em um vetor.

Entrada:

- Consumo de cada veículo.
- Preço do combustível por litro.

Importante:

- O menu deve permitir: 1. Cadastrar Consumo, 2. Estimar Custos, 0. Sair.
- Na estimativa, peça a distância a ser percorrida e use o vetor para calcular quanto cada veículo gastaria em R\$.
- Use um laço para encontrar o veículo mais econômico e outro para o menos econômico.
- Valide para que o consumo informado seja sempre > 0 e < 30 km/l.

Definição Matemática:

$$\text{Custo} = \left(\frac{\text{Distância}}{\text{Consumo}} \right) \times \text{Preço do Litro}$$

17. Uma imobiliária gerencia o aluguel de 3 prédios, onde cada prédio possui 4 apartamentos por andar de referência. Use uma matriz 3×4 para controlar a ocupação (0: Vazio, 1: Alugado).

Entrada:

- Status de cada apartamento.
- Valor do condomínio base.

Importante:

- Inicialize a matriz via menu.
- Implemente a regra: prédios com ocupação $> 75\%$ recebem um desconto de 10% no condomínio.
- Use laços aninhados para exibir graficamente a matriz (ex: [] para vazio e [X] para alugado).
- Valide para que o usuário não tente alugar um apartamento que já consta como 1.



18. Um e-commerce gerencia os preços de 3 categorias de produtos, com 4 itens em cada categoria, usando uma matriz 3×4 .

Entrada:

- Preços dos itens.
- Porcentagem de desconto de promoção relâmpago.

Importante:

- (a) O menu deve ter as opções: 1. Cadastrar Preços, 2. Aplicar Desconto, 3. Ver Preços, 0. Sair.
- (b) Valide que nenhum preço seja inserido como R\$ 0.00 ou negativo.
- (c) Ao aplicar desconto, use laços aninhados para percorrer toda a matriz e atualizar os valores.
- (d) Identifique o item mais caro de cada categoria após a promoção.

19. Um laboratório de robótica mapeia obstáculos em um grid 5×5 para testes de um pequeno robô aspirador.

Entrada:

- Tipo de terreno em cada célula (0: Piso, 1: Obstáculo, 2: Tapete).

Importante:

- (a) Preencha a matriz e valide as entradas (apenas 0, 1 ou 2 são aceitos).
- (b) Use um laço para contar quantos obstáculos (1) existem no grid.
- (c) Utilize laços aninhados para verificar se o robô ficaria "preso" (um piso cercado por 4 obstáculos).
- (d) O menu deve permitir resetar o mapa ou realizar a contagem.



20. Um sistema de RH calcula o salário final de 5 vendedores baseado em um vetor de vendas mensais e um vetor fixo de salários base.

Entrada:

- Vendas de cada vendedor.
- Valor da meta de vendas da empresa.

Importante:

- (a) Valide cada venda (deve ser ≥ 0).
- (b) Se a venda for $> 1.5 \times$ a meta, o vendedor ganha um bônus de 20%. Se for entre a meta e $1.5 \times$ a meta, bônus de 10%.
- (c) Use laços aninhados para simular uma "disputa": compare a venda de cada um com o melhor vendedor do grupo para mostrar a distância para o topo.
- (d) Exiba o faturamento total da empresa somando o vetor de vendas.

21. **Simulação de Alocação de Memória.** Um sistema operacional gerencia um segmento de memória dividido em 8 blocos. Utilize um vetor para representar o estado de cada bloco (0: Livre, 1: Alocado).

Entrada:

- Posição do bloco a ser alocado ou liberado.
- Menu (1: Alocar, 2: Liberar, 3: Ver Mapa de Memória, 0: Sair).

Importante:

- (a) Inicialize o vetor com zeros usando um laço.
- (b) No menu de Alocação, verifique se o bloco já está ocupado. Se estiver, exiba "Erro: Fragmentação detectada".
- (c) Use um laço aninhado para calcular o maior "buraco" (sequência de blocos livres contíguos) disponível na memória.
- (d) Valide para que o índice do bloco esteja entre 0 e 7. Use `while` para reentrada.



22. Tabela de Roteamento de Rede. Em uma rede com 4 roteadores, as conexões entre eles são representadas por uma matriz 4×4 contendo o custo de latência em milissegundos.

Entrada:

- Latências para a matriz `rede[4][4]`.
- Opção para verificar latência média.

Importante:

- (a) O programa deve solicitar os custos, garantindo que sejam positivos (latência não pode ser negativa).
- (b) Use laços encadeados para percorrer a matriz. Se o custo for 0, ignore (é a conexão do roteador consigo mesmo).
- (c) Identifique qual roteador possui a menor latência média para alcançar os outros 3 roteadores.
- (d) Implemente um menu que permita atualizar o custo de uma rota específica informando a Origem e o Destino.

23. Criptografia - Cifra de Substituição. Desenvolva um sistema que utilize um vetor de 5 posições para armazenar uma chave numérica de deslocamento.

Entrada:

- Chave de 5 números (ex: [3, 1, 4, 1, 5]).
- Caractere numérico a ser cifrado.

Importante:

- (a) O menu deve oferecer: 1. Definir Chave, 2. Cifrar Dados, 0. Sair.
- (b) Valide a chave: cada número deve estar entre 1 e 9.
- (c) Use um laço aninhado para aplicar a chave ciclicamente sobre uma sequência de dados, simulando a lógica de uma cifra de Vigenère simplificada.
- (d) Utilize operadores aritméticos de módulo (%) para garantir que o deslocamento não ultrapasse os limites definidos.



24. **Escalonamento de Processos (Round Robin).** Um processador gerencia 5 processos em uma fila circular (vetor). Cada posição armazena o tempo de execução (Burst Time) restante do processo.

Entrada:

- Tempos de execução de 5 processos.
- Valor do Quantum (tempo máximo por rodada).

Importante:

- (a) Use um laço `while` para simular as rodadas do processador.
- (b) Em cada rodada, use um `for` para percorrer o vetor e subtrair o Quantum do tempo de cada processo (se ele ainda for > 0).
- (c) Valide a entrada do Quantum: deve ser maior que zero.
- (d) Calcule o total de ciclos necessários para que todos os processos cheguem a zero de tempo restante.

25. **Detecção de Erros - Bit de Paridade.** Em uma transmissão de dados, uma mensagem de 16 bits é organizada em uma matriz 4×4 .

Entrada:

- Sequência de 0s e 1s para preencher a matriz.

Importante:

- (a) O sistema deve validar que apenas os números 0 ou 1 sejam inseridos.
- (b) Use laços aninhados para percorrer as linhas e calcular o "bit de paridade par" de cada uma (se a soma for ímpar, o bit de paridade é 1; se par, é 0).
- (c) Armazene os bits de paridade resultantes em um vetor separado.
- (d) Ofereça um menu para verificar se a matriz de dados está "Corrompida" ou "Íntegra".



26. Simulação de Cache LRU (Least Recently Used). Uma memória cache de 4 posições armazena IDs de páginas acessadas recentemente.

Entrada:

- ID da página solicitada pelo processador.

Importante:

- (a) O sistema deve procurar se o ID já está no vetor cache.
- (b) Se não estiver (Cache Miss), insira o ID na primeira posição livre. Se a cache estiver cheia, desloque todos os itens e insira o novo no final (FIFO simplificado).
- (c) Use um laço aninhado para reorganizar as posições do vetor após cada acesso bem-sucedido (Cache Hit), movendo o item acessado para a "frente" da lista.
- (d) Menu: 1. Acessar Página, 2. Ver Estado da Cache, 0. Sair.

27. Banco de Dados - Indexação Seqüencial. Um sistema de banco de dados armazena IDs de usuários em um vetor e seus estados (Ativo/Inativo) em uma matriz 5×2 (ID na coluna 0 e Status na coluna 1).

Entrada:

- Registros de 5 usuários.
- ID a ser pesquisado.

Importante:

- (a) Use um laço para pesquisar o ID na matriz.
- (b) Utilize condicionais para verificar o status e permitir a exclusão lógica (mudar status para 0).
- (c) O menu deve permitir: 1. Inserir, 2. Pesquisar, 3. Excluir, 0. Sair.
- (d) Valide para não permitir IDs duplicados durante a inserção.



28. Computação Gráfica - Kernel de Filtro. Simule a aplicação de um filtro de suavização simples em uma imagem de 5×5 pixels.

Entrada:

- Valores de intensidade de cinza (0 a 255) para a matriz.

Importante:

- Valide cada pixel: se fora do intervalo $[0, 255]$, peça correção.
- Para cada pixel (exceto as bordas), use laços aninhados para calcular a média dos seus 8 vizinhos imediatos.
- Gere uma nova matriz com os valores resultantes (matriz filtrada).
- O menu deve permitir ver a imagem original e a filtrada lado a lado (formatado no console).

29. Teoria da Computação - Autômato Finito. Simule um autômato que reconhece uma sequência de bits. Use uma matriz para representar a tabela de transição de estados.

Entrada:

- Estado atual e símbolo de entrada (0 ou 1).

Importante:

- Defina uma matriz onde `transicao[estado_atual][simbolo]` indica o próximo estado.
- O sistema deve receber uma sequência de 5 bits e percorrer a matriz de estados conforme a entrada.
- Ao final da sequência, verifique se o estado final é o "Estado de Aceitação".
- O menu deve permitir testar diferentes cadeias de bits.



30. **Sistemas Digitais - Tabela Verdade.** Desenvolva um programa que gere uma matriz representando a tabela verdade de uma porta lógica AND de 3 entradas.

Entrada:

- Opção de visualização da tabela.

Importante:

- Gere automaticamente a matriz 8×4 contendo todas as combinações binárias das 3 entradas ($2^3 = 8$) e o resultado da operação AND na última coluna.
- Use laços aninhados para preencher os valores binários (0 e 1) logicamente.
- Implemente uma busca por uma combinação específica informada pelo usuário via menu.
- Valide a entrada da busca (deve ser 0 ou 1 para cada uma das 3 variáveis).